

Trabajo Práctico — Unidad 3, Semana 5

Índices, vistas y vistas materializadas en Food Store

Flujo obligatorio con Kiro, OpenCode y Git

Materia	Base de Datos II
Unidad / Semana	Unidad 3 — Semana 1
Proyecto base	Food Store (continúa el esquema de las Semanas 1 a 4: schema.sql, data.sql, queries.sql)
Motor de BD	PostgreSQL 16+
Herramientas obligatorias	OpenCode (agente de codificación en terminal), Kiro (especificaciones), Git/GitHub
Modalidad	Individual o en equipos de hasta 3 alumnos
Entrega	Repositorio Git + bitácora de IA (DUIA) + informe de mediciones
Fecha de entrega	Verificar en la plataforma

1. Presentación del trabajo

El Práctico pone en juego los contenidos desarrollados en la Semana 5: la estructura y los tipos de índices, los criterios para decidir cuándo crear uno, su costo de mantenimiento, y las vistas — simples y materializadas— como herramienta de simplificación, seguridad y reutilización. El trabajo se realiza sobre el proyecto integrador Food Store, tomando como punto de partida el esquema, los datos y las consultas ya construidos en las semanas anteriores.

Como en toda la cursada, la inteligencia artificial es el motor primario del trabajo: Kiro para especificar antes de generar, y OpenCode como agente de codificación en la terminal para producir el SQL a partir de esa especificación. El repositorio Git de cada estudiante o equipo debe reflejar ese flujo de trabajo, no solo el resultado final.

2. Objetivos de aprendizaje

- Diseñar un plan de indexado para una carga de trabajo real, justificado con mediciones de EXPLAIN ANALYZE antes y después, y no solo con la intuición de que “un índice siempre ayuda”.
- Construir vistas que simplifiquen, protejan y estandarice el acceso a los datos del sistema, y una vista materializada para un reporte agregado costoso.
- Incorporar Kiro y OpenCode como flujo profesional de trabajo con IA sobre una base de datos: especificar, generar, leer línea por línea, verificar y decidir.
- Versionar en Git cada pieza del trabajo de forma que el historial de commits documente el proceso de diseño, no solo la entrega final.
- Sostener en una defensa oral cada decisión tomada: por qué se creó (*o se descartó*) cada índice, y qué garantiza cada vista.

3. Punto de partida

El trabajo continúa el esquema de Food Store construido en las semanas anteriores. Se toma como base:

- schema.sql — las cinco tablas del dominio (categoria, producto, usuario, pedido, detalle_pedido), sus tipos, restricciones y los índices ya creados en la Semana 3 (idx_producto_categoria, idx_pedido_usuario, idx_producto_nombre_vig).
- data.sql — los datos de prueba cargados en la Semana 2, que deben ampliarse para este trabajo con un volumen suficiente para que las diferencias de plan y de tiempo sean observables (*se sugiere un mínimo de varios miles de filas en pedido y detalle_pedido*).
- queries.sql — las consultas de negocio y analíticas ya resueltas en las Semanas 3 y 4, que sirven como fuente de la carga de trabajo real a indexar.

No se debe modificar el modelo de datos ni sus restricciones: el trabajo de esta semana agrega objetos —índices y vistas— sin alterar las tablas base.

4. Consignas

4.1. Parte A — Plan de indexado asistido por IA

1. A partir de queries.sql, identificar al menos **tres consultas frecuentes** del sistema que hoy resuelven con Seq Scan sobre una tabla de tamaño considerable.
2. Para cada una, redactar en Kiro una especificación breve y precisa de lo que se necesita optimizar: la consulta, la frecuencia con la que se ejecuta, y qué columnas participan del filtro, del JOIN o del ORDER BY.
3. A partir de esa especificación, usar OpenCode para proponer el índice adecuado (*tipo, columnas y, si corresponde, orden en un índice compuesto o condición de un índice parcial*).
4. Antes de crear cada índice propuesto, ejecutar EXPLAIN ANALYZE sobre la consulta y registrar el plan y el tiempo. Crear el índice y volver a medir. El cambio de plan (de Seq Scan a Index Scan o Bitmap Heap Scan) y la mejora de tiempo deben quedar documentados.
5. Medir también el costo de los índices creados sobre las escrituras: comparar el tiempo de una carga de varios cientos de INSERT en detalle_pedido antes y después de sumar los nuevos índices.
6. De las propuestas recibidas de la IA, descartar explícitamente al menos una por **sobreindexación** (por ejemplo, un índice redundante con otro ya existente, o sobre una columna de baja cardinalidad sin condición parcial), justificando la decisión por escrito.

Entregable de la Parte A

indices.sql (las sentencias CREATE INDEX finalmente aceptadas, comentadas) e informe_mediciones.md con el EXPLAIN ANALYZE de cada consulta antes y después, el tiempo de la carga de escritura antes y después, y la justificación del índice descartado.

4.2. Parte B — Vistas para los reportes del sistema

1. Especificar en Kiro al menos tres vistas necesarias para los reportes habituales de Food Store: productos vigentes con su categoría, pedidos con los datos del usuario, y detalle de un pedido con el nombre del producto. La especificación debe indicar con precisión las columnas a exponer, el filtro de vigencia y, si corresponde, qué columna debe ocultarse por seguridad.
2. Generar las vistas con OpenCode a partir de esa especificación.

3. Verificar la equivalencia de resultados: para cada vista, ejecutar también la consulta equivalente escrita por el propio estudiante y comprobar que ambos resultados coinciden exactamente antes de dar la vista por válida.
4. Al menos una vista debe aplicar el criterio de seguridad visto en la teoría: exponer usuario sin la columna contraseña, de modo que pueda otorgarse SELECT sobre esa vista sin dar acceso a la tabla base.

Entregable de la Parte B

views.sql con las vistas creadas, y una nota en informe_mediciones.md o en la bitácora que documente la verificación de equivalencia de cada una contra la consulta manual.

4.3. Parte C — Una vista materializada

1. Elegir un reporte agregado costoso del sistema (por ejemplo, facturación por categoría y mes, u otro que el estudiante identifique a partir de las consultas analíticas de la Semana 4) y crear la vista materializada correspondiente, con WITH DATA y un índice único que permita, a futuro, un REFRESH CONCURRENTLY.
2. Medir el tiempo de consultar el reporte contra la vista materializada frente al tiempo de ejecutar la consulta original sin materializar.
3. Documentar con qué frecuencia debería ejecutarse el REFRESH MATERIALIZED VIEW dado el uso esperado del reporte, y qué implica para los usuarios que el dato no se actualice en cada REFRESH.

Entregable de la Parte C

La vista materializada y su índice, agregados a views.sql (o en un archivo materializadas.sql aparte), con la medición de tiempo y la justificación de la frecuencia de refresco en el informe.

5. Flujo de trabajo obligatorio: Kiro, OpenCode y Git

Esta cátedra exige que el proceso de trabajo con IA quede documentado y sea verificable, no solo el resultado. Se debe seguir esta secuencia para cada pieza del trabajo (cada índice, cada vista, la vista materializada):

1. Especificar primero en Kiro: qué se necesita, sobre qué consulta o reporte, con qué criterio de aceptación. El archivo de especificación se conserva en el repositorio (carpeta specs/).
2. Generar con OpenCode a partir de esa especificación, como agente de codificación en la terminal. Ningún script generado se ejecuta sin haber leído y comprendido línea por línea.

3. Probar sobre una copia de la base o dentro de una transacción reversible, con respaldo previo cuando corresponda, siguiendo el protocolo de seguridad de la cátedra ya utilizado desde la Unidad 1.
4. Versionar en Git: cada índice, cada vista y la vista materializada quedan en commits separados y descriptivos (por ejemplo: git commit -m "Índice compuesto pedido(fecha, estado) — reduce Seq Scan en reporte mensual"). El historial debe permitir mostrar, en la defensa oral, el diff de cada commit.

Ejemplo de especificación mínima en Kiro (spec) para una pieza de este trabajo

```
# spec: indice_pedido_fecha_estado
Objetivo: acelerar el reporte "pedidos confirmados de un mes".
Consulta afectada: SELECT id, fecha, total FROM pedido
                    WHERE fecha BETWEEN :desde AND :hasta
                    AND estado = 'CONFIRMADO' AND eliminado =
FALSE;
Columnas candidatas: fecha (alta selectividad), estado (baja
selectividad).
Criterio de aceptación: el plan pasa de Seq Scan a Index/Bitmap
Scan y el tiempo baja al menos un orden de magnitud.
```

Este spec es lo que se le entrega a OpenCode como contexto: no se le pide “creame un índice” sin más, sino que se le da el objetivo, la consulta exacta y el criterio con el que se va a aceptar o rechazar su propuesta. Esa es la diferencia entre delegar la escritura del SQL y delegar la decisión: la decisión, dice el programa de la materia, nunca se delega.

6. Declaración de Uso de IA (DUIA) y bitácora

El trabajo debe incluir un archivo duia.md que registre, para cada interacción relevante con Kiro u OpenCode:

- Qué herramienta se usó y con qué propósito (especificar, generar, revisar).
- El prompt o el spec utilizado, tal como se le entregó a la herramienta.
- Qué propuso la IA (el índice, la vista, la explicación de un plan).
- Qué se aceptó, qué se modificó y qué se descartó, con la justificación técnica de la decisión.

La bitácora debe incluir, como mínimo, el caso de sobreindexación descartado en la Parte A y la verificación de equivalencia de resultados de al menos una vista de la Parte B.

7. Entregables y formato de entrega

El trabajo se entrega como un repositorio Git (enlace público o acceso otorgado a la cátedra) con la siguiente estructura mínima:

```
food-store/
├── schema.sql           (heredado, sin modificar)
├── data.sql             (heredado, ampliado si hace falta
volumen)
├── queries.sql          (heredado)
├── indices.sql          (nuevo – Parte A)
├── views.sql            (nuevo – Partes B y C)
├── specs/               (especificaciones de Kiro)
├── duia.md              (bitácora de uso de IA)
├── informe_mediciones.md (EXPLAIN ANALYZE antes/después,
lectura y escritura)
└── README.md            (cómo reproducir las pruebas de
este trabajo)
```

El historial de commits del repositorio es parte de la entrega: se evalúa que exista, sea legible y corresponda al flujo descrito en el Punto 5, no solo el estado final de los archivos.

8. Defensa oral

Regla de la cátedra

Cada estudiante debe poder explicar y justificar, sin apoyo de IA en el momento, cualquier índice o vista entregado: por qué se creó, qué plan cambió, qué costo tiene en la escritura, y por qué se descartó lo que se descartó. Entregar un script que no se puede explicar en la defensa equivale a no haber realizado el trabajo, según establece el programa de la materia.

9. Criterios de evaluación (orientativos)

10. Cierre

Al finalizar este trabajo, el proyecto integrador Food Store cuenta con un plan de indexado justificado con datos y con el conjunto de vistas que van a servir de base para los objetos programables —procedimientos, funciones y disparadores— que se desarrollan en la Semana 6. Se recomienda no eliminar ni reescribir los archivos de esta entrega: van a reutilizarse en la próxima.